

使用 Dataproc 和 Cloud Pub/Sub 建立變更資料擷取機制

來源：<https://codelabs.developers.google.com/cdc-using-dataproc-cloud-pubsub?hl=zh-tw>

步驟數：8

在本程式碼研究室中，您將使用 Google Cloud Pub/Sub 和 Dataproc 建構即時資料管道，以便實作變更資料擷取 (CDC)

目錄

1. [簡介](#)
2. [設定 Cloud Pub/Sub](#)
3. [模擬資料庫變更](#)
4. [為 Dataproc 建立服務帳戶](#)
5. [建立 Dataproc 叢集](#)
6. [將 Spark 工作提交至 Dataproc 叢集](#)
7. [清除所用資源](#)
8. [恭喜](#)

1. 簡介



上次更新時間：2025 年 6 月 19 日

什麼是變更資料擷取？

變更資料擷取 (CDC) 是一組軟體設計模式，用於判斷及追蹤資料庫中變更的資料。簡單來說，這是一種擷取及記錄資料變更的方式，以便將這些變更複製到其他系統。

變更資料擷取 (CDC) 功能在各種資料導向情境中都非常實用，例如資料遷移、即時資料倉儲和分析、災難復原和高可用性、稽核和法規遵循等。

資料遷移

CDC 支援增量資料傳輸，可減少停機時間並盡量降低中斷情形，簡化資料遷移專案。

即時資料倉儲和分析

CDC 可確保資料倉儲和分析系統，持續更新營運資料庫的最新異動。

讓企業能根據即時資訊做出決策。

災難復原和高可用性

變更資料擷取功能可將資料即時複製到次要資料庫，以利災難復原。如果發生故障，CDC 可快速容錯移轉至次要資料庫，盡量縮短停機時間並減少資料遺失。

稽核與法規遵循

CDC 提供詳細的資料變更稽核追蹤記錄，這對法規遵循和監管要求至關重要。

建構項目

在本程式碼研究室中，您將使用 Cloud Pub/Sub、Dataproc、Python 和 Apache Spark，建構變更資料擷取 (CDC) 資料管道。管道會執行下列動作：

- 模擬資料庫變更，並以事件形式發布至可擴充且可靠的訊息傳遞服務 Cloud Pub/Sub。
- 運用 Google Cloud 的代管 Spark 和 Hadoop 服務 Dataproc 的強大功能，即時處理這些事件。

連結這些服務後，您就能建立強大的管道，擷取及處理資料變更，為即時分析、資料倉儲和其他重要應用程式奠定基礎。

課程內容

- 如何建立基本變更資料擷取管道
- [Dataproc](#) 串流處理
- [Cloud Pub/Sub](#)，用於即時訊息傳遞
- [Apache Spark](#) 基礎知識

本程式碼研究室著重於 [Dataproc](#) 和 [Cloud Pub/Sub](#)。我們不會對與主題無關的概念和程式碼多做介紹，但會事先準備好這些程式碼區塊，屆時您只要複製及貼上即可。

軟硬體需求

- 已設定專案的有效 GCP 帳戶。如果沒有帳戶，可以申請免費試用。
- 安裝並設定 [gcloud CLI](#)。
- 安裝 Python 3.7 以上版本，用於模擬資料庫變更及與 Pub/Sub 互動。
- [Dataproc](#)、[Cloud Pub/Sub](#)、[Apache Spark](#) 和 [Python](#) 的基本知識。

事前準備

在終端機中執行下列指令，啟用必要的 API：

□□□

```
gcloud services enable \  
  dataproc.googleapis.com \  
  pubsub.googleapis.com \  

```

步驟 2 · 約 0 分鐘

2. 設定 Cloud Pub/Sub

建立主題

這個主題會用於發布資料庫變更。Dataproc 作業會成為這些訊息的消費者，並處理訊息以擷取變更資料。如要進一步瞭解主題，請參閱[這份官方說明文件](#)。

```
gcloud pubsub topics create database-changes
```

建立訂閱項目

建立訂閱項目，用於取用 Pub/Sub 中的訊息。如要進一步瞭解訂閱項目，請參閱[這份官方說明文件](#)。

```
gcloud pubsub subscriptions create --topic database-changes change-subscriber
```

注意：稍後會使用這個訂閱項目查看已發布的訊息。

步驟 3 · 約 0 分鐘

3. 模擬資料庫變更

步驟

1. 建立 Python 指令碼 (例如 `simulate_cdc.py`)，模擬資料庫變更並發布至 Pub/Sub。

```
from google.cloud import pubsub_v1
import json
import time
import random

project_id = "YOUR_PROJECT_ID" # Replace with your GCP project ID
topic_id = "database-changes"

publisher = pubsub_v1.PublisherClient()
topic_path = publisher.topic_path(project_id, topic_id)

def publish_message(data):
    data_str = json.dumps(data).encode("utf-8")
    future = publisher.publish(topic_path, data=data_str)
    print(f"Published message ID: {future.result()}")

def simulate_change():
    change_types = ["INSERT", "UPDATE", "DELETE"]
    change_type = random.choice(change_types)
    record_id = random.randint(1, 100)
    timestamp = time.time()
    change_data = {
        "change_type": change_type,
        "record_id": record_id,
        "timestamp": timestamp,
        "data": {"field1": "value1", "field2": "value2"}, #Place holder data.
    }
    publish_message(change_data)

if __name__ == "__main__":
    while True:
        simulate_change()
        time.sleep(2) # Simulate changes every 2 seconds
```

將 `YOUR_PROJECT_ID` 替換為實際的 GCP 專案 ID

2. 安裝 Pub/Sub 用戶端程式庫：

```
pip install google-cloud-pubsub
```

3. 在終端機中執行指令碼。這個指令碼會持續執行，並每 2 秒將訊息發布至 Pub/Sub 主題。

```
python simulate_cdc.py
```

4. 執行指令碼 1 分鐘後，Pub/Sub 中就會有足夠的訊息可供取用。視作業系統而定，按下 Ctrl + C 或 Cmd + C 即可終止執行的 Python 指令碼。
5. 查看已發布的訊息：

開啟另一個終端機，然後執行下列指令來查看發布的訊息：

```
gcloud pubsub subscriptions pull --auto-ack change-subscriber
```

您應該會看到包含訊息和其他欄位的資料表列：

```
{"change_type": "UPDATE", "record_id": 10, "timestamp": 1742466264.888465, "data": {"field1": "value1", "field2": "value2"}}
```

說明

- Python 指令碼會隨機產生 **INSERT**、**UPDATE** 或 **DELETE** 事件，模擬資料庫變更。
 - 每項變更都會以 JSON 物件表示，其中包含變更類型、記錄 ID、時間戳記和資料。
 - 這項指令碼會使用 Cloud Pub/Sub 用戶端程式庫，將這些變更事件發布至 **database-changes** 主題。
 - 訂閱者指令可讓您查看傳送至 Pub/Sub 主題的訊息。
-

4. 為 Dataproc 建立服務帳戶

在本節中，您將建立 Dataproc 叢集可使用的服務帳戶。您也可以指派必要的權限，允許叢集執行個體存取 Cloud Pub/Sub 和 Dataproc。

1. 建立服務帳戶：

```
export SERVICE_ACCOUNT_NAME="dataproc-service-account"

gcloud iam service-accounts create $SERVICE_ACCOUNT_NAME
```

2. 新增 Dataproc 工作站角色，允許服務帳戶建立叢集並執行工作。在下列指令中，將上一個指令產生的服務帳戶 ID 新增為成員：

```
export PROJECT=$(gcloud info --format='value(config.project)')

gcloud projects add-iam-policy-binding $PROJECT \
  --role roles/dataproc.worker \
  --member="serviceAccount:<your-service-account-with-domain>"
```

3. 新增 Pub/Sub 訂閱者角色，允許服務帳戶訂閱「change-subscriber」Pub/Sub 訂閱：

```
gcloud beta pubsub subscriptions add-iam-policy-binding \
  change-subscriber \
  --role roles/pubsub.subscriber \
  --member="serviceAccount:<your-service-account-with-domain>"
```

步驟 5 · 約 0 分鐘

5. 建立 Dataproc 叢集

Dataproc 叢集會執行 Spark 應用程式，處理 Pub/Sub 中的訊息。您需要使用上一節建立的服務帳戶。Dataproc 會將這個服務帳戶指派給叢集中的每個執行個體，讓所有執行個體都取得執行應用程式的正確權限。

使用下列指令建立 Dataproc 叢集：

```
gcloud dataproc clusters create cdc-dataproc-cluster \  
  --region=us-central1 \  
  --zone=us-central1-a \  
  --scopes=pubsub,datastore \  
  --image-version=1.3 \  
  --service-account="<your-service-account-with-domain-id>"
```

步驟 6 · 約 0 分鐘

6. 將 Spark 工作提交至 Dataproc 叢集

Spark 串流應用程式會處理 Pub/Sub 中的資料庫變更訊息，並將訊息列印至控制台。

步驟

1. 建立目錄，並將消費者的原始碼新增至 PubsubConsumer.scala 檔案

```
mkdir -p dataproc-pubsub-spark-streaming/spark/src/main/scala/demo && \  
touch dataproc-pubsub-spark-streaming/spark/src/main/scala/demo/PubsubConsumer.scala
```

```

package demo

import java.nio.charset.StandardCharsets
import org.apache.spark.SparkConf
import org.apache.spark.storage.StorageLevel
import org.apache.spark.streaming.dstream.DStream
import org.apache.spark.streaming.pubsub.{PubsubUtils, SparkGCPCredentials}
import org.apache.spark.streaming.{Seconds, StreamingContext}

object PubsubConsumer {

  def createContext(projectID: String, checkpointDirectory: String)
    : StreamingContext = {

    // [START stream_setup]
    val sparkConf = new SparkConf().setAppName("DatabaseUpdates")
    val ssc = new StreamingContext(sparkConf, Seconds(1))

    // Set the checkpoint directory
    val yarnTags = sparkConf.get("spark.yarn.tags")
    val jobId = yarnTags.split(",").filter(_.startsWith("dataproc_job")).head
    ssc.checkpoint(checkpointDirectory + '/' + jobId)

    // Create stream
    val messagesStream: DStream[String] = PubsubUtils
      .createStream(
        ssc,
        projectID,
        None,
        "change-subscriber", // Cloud Pub/Sub subscription for incoming database updates
        SparkGCPCredentials.builder.build(), StorageLevel.MEMORY_AND_DISK_SER_2)
      .map(message => new String(message.getData(), StandardCharsets.UTF_8))
    // [END stream_setup]

    processStringDStream(messagesStream)

    ssc
  }

  def processStringDStream(stringDStream: DStream[String]): Unit = {
    stringDStream.foreachRDD { rdd =>
      if (!rdd.isEmpty()) {
        val listOfStrings: List[String] = rdd.collect().toList
        listOfStrings.foreach(str => println(s"message received: $str"))
      } else {
        println("looking for message...")
      }
    }
  }

  def main(args: Array[String]): Unit = {
    if (args.length != 2) {

```

```
    System.err.println("arguments are not passed correctly!")
    System.exit(1)
  }

  val Seq(projectID, checkpointDirectory) = args.toSeq

  // Create Spark context
  val ssc = StreamingContext.getOrCreate(checkpointDirectory,
    () => createContext(projectID, checkpointDirectory))

  // Start streaming until we receive an explicit termination
  ssc.start()
  ssc.awaitTermination()
}

}
```

2. 建立 pom.xml 並新增下列內容

```
touch dataproc-pubsub-spark-streaming/spark/pom.xml
```

```
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 http://maven.apache.org/xsd/maven-
4.0.0.xsd">
  <modelVersion>4.0.0</modelVersion>

  <properties>
    <maven.compiler.source>1.8</maven.compiler.source>
    <maven.compiler.target>1.8</maven.compiler.target>
    <jvm.options.xms>-Xms512M</jvm.options.xms>
    <jvm.options.xmx>-Xmx2048M</jvm.options.xmx>
    <jvm.options.maxpermsize>-XX:MaxPermSize=2048M</jvm.options.maxpermsize>
    <jvm.options.xx>-XX:+CMSClassUnloadingEnabled</jvm.options.xx>
  </properties>

  <groupId>dataproc-spark-demos</groupId>
  <artifactId>spark-streaming-pubsub-demo</artifactId>
  <version>1.0-SNAPSHOT</version>

  <dependencies>

    <dependency>
      <groupId>org.apache.spark</groupId>
      <artifactId>spark-streaming_2.11</artifactId>
      <version>2.2.0</version>
      <scope>provided</scope>
    </dependency>

    <dependency>
      <groupId>com.google.cloud</groupId>
      <artifactId>google-cloud-datastore</artifactId>
      <version>1.34.0</version>
    </dependency>

    <dependency>
      <groupId>org.apache.bahir</groupId>
      <artifactId>spark-streaming-pubsub_2.11</artifactId>
      <version>2.2.0</version>
    </dependency>

    <dependency>
      <groupId>org.scalacheck</groupId>
      <artifactId>scalacheck_2.11</artifactId>
      <version>1.14.0</version>
      <scope>test</scope>
    </dependency>

  </dependencies>

  <build>
    <plugins>
```

```

<plugin>
  <groupId>net.alchim31.maven</groupId>
  <artifactId>scala-maven-plugin</artifactId>
  <version>3.3.2</version>
  <executions>
    <execution>
      <id>compile</id>
      <goals>
        <goal>compile</goal>
      </goals>
      <phase>compile</phase>
    </execution>
    <execution>
      <id>test-compile</id>
      <goals>
        <goal>testCompile</goal>
      </goals>
      <phase>test-compile</phase>
    </execution>
    <execution>
      <phase>process-resources</phase>
      <goals>
        <goal>compile</goal>
      </goals>
    </execution>
  </executions>
</plugin>

```

```

<plugin>
  <groupId>org.apache.maven.plugins</groupId>
  <artifactId>maven-shade-plugin</artifactId>
  <version>3.1.1</version>
  <executions>
    <execution>
      <phase>package</phase>
      <goals>
        <goal>shade</goal>
      </goals>
      <configuration>
        <transformers>
          <transformer
implementation="org.apache.maven.plugins.shade.resource.ManifestResourceTransformer">
            <mainClass>demo.PubsubConsumer</mainClass>
          </transformer>
        </transformers>
        <relocations>
          <relocation>
            <pattern>com</pattern>
            <shadedPattern>repackaged.com</shadedPattern>
          </relocation>
        </relocations>
        <includes>
          <include>com.google.protobuf.**</include>

```

```

        <include>com.google.common.**</include>
    </includes>
    </relocation>
    </relocations>
    </configuration>
    </execution>
    </executions>
    </plugin>

    </plugins>
</build>

</project>

```

3. 變更為專案的 Spark 目錄，並將路徑儲存至環境變數以供後續使用：

```

cd dataproc-pubsub-spark-streaming
export REPO_ROOT=$PWD

```

4. 變更目錄：

```

cd $REPO_ROOT/spark

```

5. 下載 Java 1.8，然後將資料夾放在 /usr/lib/jvm/ 中。然後將 JAVA_HOME 變更為指向這個位置：

```

export JAVA_HOME=/usr/lib/jvm/<your-java-1.8>/jre

```

6. 建構應用程式 JAR 檔

```

mvn clean package

```

系統會在 `spark/target` 目錄中建立含有應用程式程式碼與依附元件的 `spark-streaming-pubsub-demo-1.0-SNAPSHOT.jar` 封存

7. 提交 Spark 應用程式：

```
export PROJECT=$(gcloud info --format='value(config.project)')
export JAR="spark-streaming-pubsub-demo-1.0-SNAPSHOT.jar"
export
SPARK_PROPERTIES="spark.dynamicAllocation.enabled=false,spark.streaming.receiver.writeAheadLog.enabled=true"
export ARGUMENTS="$PROJECT hdfs:///user/spark/checkpoint"

gcloud dataproc jobs submit spark \
  --cluster cdc-dataproc-cluster \
  --region us-central1 \
  --async \
  --jar target/$JAR \
  --max-failures-per-hour 10 \
  --properties $SPARK_PROPERTIES \
  -- $ARGUMENTS
```

8. 顯示執行中的工作清單，並記下工作的 `JOB_ID` 值：

```
gcloud dataproc jobs list --region=us-central1 --state-filter=active
```

輸出內容類似如下

JOB_ID	TYPE	STATUS
473ecb6d14e2483cb88a18988a5b2e56	spark	RUNNING

9. 在瀏覽器中開啟下列網址，以查看工作輸出。將 `[JOB_ID]` 換成您上一步記下的值。

```
https://console.cloud.google.com/dataproc/jobs/\[JOB\_ID\]?region=us-central1
```

10. 輸出結果會與下列內容相似：


```
looking for message...
looking for message...
message received: {"change_type": "INSERT", "record_id": 72, "timestamp": 1745409434.969086,
"data": {"field1": "value1", "field2": "value2"}}
message received: {"change_type": "INSERT", "record_id": 55, "timestamp": 1745409439.269171,
"data": {"field1": "value1", "field2": "value2"}}
looking for message...
message received: {"change_type": "DELETE", "record_id": 71, "timestamp": 1745409430.673305,
"data": {"field1": "value1", "field2": "value2"}}
message received: {"change_type": "UPDATE", "record_id": 15, "timestamp": 1745409432.819154,
"data": {"field1": "value1", "field2": "value2"}}
message received: {"change_type": "DELETE", "record_id": 18, "timestamp": 1745409426.3570209,
"data": {"field1": "value1", "field2": "value2"}}
message received: {"change_type": "INSERT", "record_id": 85, "timestamp": 1745409428.5078359,
"data": {"field1": "value1", "field2": "value2"}}
message received: {"change_type": "UPDATE", "record_id": 18, "timestamp": 1745409441.436026,
"data": {"field1": "value1", "field2": "value2"}}
looking for message...
```

在 Dataproc 中執行的 Spark 串流工作會從 Pub/Sub 提取訊息、處理訊息，並在控制台中顯示輸出內容。

11. 終止工作：執行下列指令來終止工作。將 JOB_ID 換成我們稍早記下的 ID

```
gcloud dataproc jobs kill [JOB_ID] --region us-central1 --quiet
```

恭喜！您剛才建立的 CDC 管道功能強大，可擷取 Pub/Sub 中的資料庫變更，並使用在 Cloud Dataproc 中執行的 Spark 串流處理這些變更。

7. 清除所用資源

清除您建立的任何資源，這樣您日後就不需為其付費。如要避免付費，最簡單的方法就是刪除您針對教學課程建立的專案。或者，您也可以刪除個別資源。

執行下列指令，刪除個別資源

```
gcloud dataproc clusters delete cdc-dataproc-cluster --quiet
gcloud pubsub topics delete database-changes --quiet
gcloud pubsub subscriptions delete change-subscriber --quiet
gcloud iam service-accounts delete <your-service-account-with-domain> --quiet
```

8. 恭喜

恭喜！您已完成實作程式碼研究室，瞭解如何使用 Google Cloud Platform 建構強大的即時資料管道。現在來回顧您學到的內容：

- **模擬變更資料擷取 (CDC)：**您已瞭解 CDC 的基本概念，並實作 Python 指令碼來模擬資料庫變更，產生代表即時資料修改的事件。
- **運用 Cloud Pub/Sub：**您設定 Cloud Pub/Sub 主題和訂閱項目，為串流 CDC 事件提供可擴充且可靠的訊息傳遞服務。您已將模擬資料庫變更發布至 Pub/Sub，建立即時資料串流。
- **使用 Dataproc 和 Spark 處理資料：**您已佈建 Dataproc 叢集，並部署 Spark Streaming 工作，從 Pub/Sub 訂閱項目取用訊息。您已即時處理及轉換傳入的 CDC 事件，並在 Dataproc 工作記錄中顯示結果。
- **建構端對端即時管道：**您已成功整合這些服務，建立完整的資料管道，即時擷取、串流及處理資料變更。您已實際體驗如何建構可處理連續資料串流的系統。
- **使用 Spark Pub/Sub 連接器：**您已成功設定 Dataproc 叢集，使用 Spark Pub/Sub 連接器，這是 Spark Structured Streaming 從 Pub/Sub 讀取資料的必要條件。

您現在已具備穩固的基礎，可為各種應用程式 (包括即時分析、資料倉儲和微服務架構) 建構更複雜精密的資料管道。繼續探索及建構！

參考文件

- [Dataproc 簡介](#)
 - [Cloud Pub/Sub 總覽](#)
 - [提交 Dataproc 工作](#)
-